

ASSIGNMENT NO. 1

NAME OF THE STUDENT: Chudasama Dhruvin Hitesh
CLASS: Comps

ROLL NO.: 14
DIVISION: A

LEARNING OBJECTIVE	To learn the basic concepts of object-oriented programming
LEARNING OUTCOME	Apply fundamental programming constructs like branching and looping
COURSE OUTCOME	Students will be able to apply the fundamental programming constructs.
PROGRAM OUTCOME	PO1,PO2,PO3
BLOOM'S TAXONOMY LEVEL	Remember, Apply, Create.
TASK	1. Explain Object oriented programming features in details. OOP is a programming paradigm based on the concept of objects that contain data and methods. The main features of OOP are: --Encapsulation: This is the bundling of data (variables) and methods (functions) that operate on the data into a single unit or class. It restricts direct access to some components, which means the internal representation of an object is hidden from the outside world. This is typically done through access modifiers like private, protected, and public. Example: <pre> class Person { private String name; // data encapsulated public void setName(String name) { this.name = name; } public String getName() { return name; } } </pre>

```
}  
}
```

--**Inheritance**: This allows one class to inherit the properties and behavior of another class. The class that inherits is called a child or subclass, and the class being inherited from is the parent or superclass. It promotes code reuse.

Example:

```
class Animal {  
    void eat() {  
        System.out.println("This animal eats food.");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("The dog barks.");  
    }  
}
```

--**Polymorphism**: Polymorphism allows objects of different classes to be treated as objects of a common superclass. It can be achieved through method overriding (runtime polymorphism) and method overloading (compile-time polymorphism).

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

// Polymorphism at play:
Animal obj = new Dog();
obj.sound(); // Output: Dog barks

--**Abstraction**: Abstraction means representing essential features without including background details. In Java, abstraction is achieved through abstract classes and interfaces.

Example:

```
abstract class Vehicle {  
    abstract void start(); // Abstract method (no body)  
}  
  
class Car extends Vehicle {  
    void start() {  
        System.out.println("Car starts");  
    }  
}
```

2. Why JVM is important in java.

JVM (Java Virtual Machine) is an integral part of the Java platform. It is responsible for the following:

-Platform Independence: Java is "write once, run anywhere" because of the JVM. Java code is compiled into bytecode, which can run on any platform that has a JVM, making Java platform-independent.

-Memory Management: JVM handles memory allocation and garbage collection automatically, reducing memory leaks and ensuring better management of resources.

-Security: JVM provides security features such as bytecode verification and sandboxing, ensuring that Java programs are safe to execute.

-Performance: JVM includes features like Just-In-Time (JIT) compilation, which converts bytecode into machine code at runtime, improving the performance of Java applications.

3. Explain Static use in java.

In Java, static is a keyword used to share variables or methods among all instances of a class. It is mainly used for memory management.

--Static Variables: A static variable belongs to the class, not to any specific instance. All instances share this variable, and its value remains constant across all instances.

Example:

```
class Example {  
    static int count = 0; // static variable  
  
    Example() {  
        count++;  
        System.out.println(count);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Example obj1 = new Example(); // Output: 1  
        Example obj2 = new Example(); // Output: 2  
    }  
}
```

--Static Methods: A static method belongs to the class rather than any instance. It can be called without creating an object of the class. Static methods cannot access instance variables or instance methods.

Example:

```
class Example {  
    static void display() {  
        System.out.println("This is a static method.");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Example.display(); // No need to create an object  
    }  
}
```

--Static Block: A static block is used to initialize static variables. It gets executed when the class is loaded into memory.

Example:

```
class Example {  
    static int num;  
  
    static {  
        num = 10;  
    }  
}
```

```
        System.out.println("Static block executed.");
    }

    public static void main(String[] args) {
        System.out.println("Main method executed.");
    }
}
```

4. What is the package? Explain in detail about

A package in Java is a namespace that organizes a set of related classes and interfaces. It helps avoid name conflicts and can contain classes, interfaces, enums, and sub-packages. Packages can be classified into two types:

Built-in Packages: These are predefined packages provided by Java, like java.util, java.io, java.lang, etc.

User-defined Packages: These are packages created by the user to organize their classes.

Purpose of Using Packages:

Namespace Management: Packages prevent naming conflicts by organizing classes into namespaces. For example, if two classes have the same name, they can still exist in different packages.

Modularity: It helps to group related classes together, increasing the modularity of the application.

Access Control: Packages also play a role in defining access levels. For instance, classes in the same package can access each other's members that have package-private visibility.

Creating and Using Packages: To create a package, use the package keyword at the top of the file.

Example:

```
package com.mycompany.myapp;

public class MyClass {
    public void display() {
        System.out.println("Hello from MyClass in
```

	<pre>com.mycompany.myapp"); } }</pre> To use a package in another class, import it: <pre>import com.mycompany.myapp.MyClass;</pre> <pre>class Main { public static void main(String[] args) { MyClass obj = new MyClass(); obj.display(); } }</pre> Java also supports sub-packages, which are packages inside another package. These help further categorize classes for larger projects.
REFERENCES	